

PORTADA

Técnicas de Programación

Unidad 3 — Técnicas de Programación: Principios SOLID

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

Cambias una línea en una clase y, sin saber por qué, se rompen diez módulos que nunca deberían haberse enterado de que esa clase existía.



TRANSICIÓN

De construir objetos a diseñarlos bien

En la Unidad 2 aprendiste a construir clases, herencia, interfaces y polimorfismo: las **herramientas** del paradigma objetual. La Unidad 3 se enfoca en algo distinto: **cómo usar bien esas herramientas** para que el código sea fácil de mantener, extender y probar a medida que un proyecto crece. Los **principios SOLID** son cinco reglas de diseño, formuladas por Robert C. Martin a partir de ideas previas de Bertrand Meyer y Barbara Liskov, que ayudan justamente a eso.

CONCEPTO

SOLID: cinco letras, cinco principios

SOLID es un acrónimo que agrupa cinco principios de diseño orientado a objetos. Ninguno es una regla sintáctica de Java: son criterios para decidir **cómo dividir responsabilidades entre clases e interfaces**.

Letra	Principio
S	Single Responsibility — Responsabilidad única
O	Open/Closed — Abierto/cerrado
L	Liskov Substitution — Sustitución de Liskov
I	Interface Segregation — Segregación de interfaces
D	Dependency Inversion — Inversión de dependencias

CONCEPTO — S

Responsabilidad única: una clase, un motivo de cambio

Una clase debe tener **un solo motivo para cambiar**. Si una clase calcula, guarda en archivo y además imprime un reporte, tiene tres motivos distintos para cambiar (regla de negocio, formato de archivo, formato de impresión) — y cambiar uno arriesga romper los otros dos sin que se note.

```
// Viola SRP: mezcla cálculo, persistencia e impresión
public class Factura {
    public double calcularTotal() { /* ... */ }
    public void guardarEnArchivo() { /* ... */ }
    public void imprimirReporte() { /* ... */ }
}
```

CONCEPTO — S

Separando responsabilidades

La solución no es eliminar funcionalidad, es **repartirla** en clases distintas, cada una con un único motivo de cambio.

```
public class Factura {  
    public double calcularTotal() { /* ... */ }  
}  
  
public class RepositorioFacturas {  
    public void guardar(Factura f) { /* ... */ }  
}  
  
public class ReporteFactura {  
    public void imprimir(Factura f) { /* ... */ }  
}
```

Ahora un cambio en el formato de impresión no toca la clase que calcula el total.

CONCEPTO — 0

Abierto/cerrado: extender sin modificar

Una clase debe estar **abierta a extensión** (se le puede agregar comportamiento nuevo) pero **cerrada a modificación** (no hay que tocar su código ya probado para lograrlo). El mecanismo típico en Java para esto son las **interfaces y la herencia**: en vez de agregar un nuevo `if` a un método existente, se agrega una clase nueva que implementa el mismo contrato.

CONCEPTO — 0

De un switch que crece a una interfaz que se extiende

```
// Cada descuento nuevo obliga a MODIFICAR este método
public double calcularDescuento(String tipoCliente, double monto) {
    if (tipoCliente.equals("VIP")) return monto * 0.20;
    if (tipoCliente.equals("FRECUENTE")) return monto * 0.10;
    // ...un "else if" más por cada tipo de cliente nuevo
    return 0;
}
```

```
public interface Descuento {
    double calcular(double monto);
}

public class DescuentoVip implements Descuento {
    public double calcular(double monto) { return monto * 0.20; }
}

// Un cliente nuevo = una clase nueva, sin tocar las anteriores
```


CONCEPTO — L

Sustitución de Liskov: la subclase no puede sorprender

Si una clase `B` hereda de `A`, cualquier parte del programa que use un objeto de tipo `A` debe poder recibir un objeto de tipo `B` **sin darse cuenta del cambio y sin que el programa se rompa**. La subclase puede especializar comportamiento, pero no puede violar las expectativas que ya generaba la clase padre (por ejemplo, lanzando una excepción donde el padre nunca lo hacía).

CONTRA EJEMPLO

El clásico caso del rectángulo y el cuadrado

```
public class Rectangulo {  
    protected double ancho, alto;  
    public void setAncho(double a) { this.ancho = a; }  
    public void setAlto(double a) { this.alto = a; }  
}  
  
public class Cuadrado extends Rectangulo {  
    public void setAncho(double a) { ancho = a; alto = a; } // isorpesa!  
    public void setAlto(double a) { ancho = a; alto = a; }  
}
```

Un código que espera un `Rectangulo` y hace `setAncho(5); setAlto(2)` asume que el área queda en 10 — pero si recibe un `Cuadrado`, el área termina en 4. `Cuadrado` no puede sustituir a `Rectangulo` sin romper esa expectativa: viola LSP.



CONCEPTO — I

Segregación de interfaces: contratos pequeños y específicos

Ninguna clase debería verse obligada a implementar métodos que **no necesita**. Una interfaz grande con muchos métodos ("interfaz gorda") fuerza a las clases que la implementan a escribir métodos vacíos o que lanzan excepciones — señal de que el contrato debería dividirse en interfaces más pequeñas y específicas.

```
// Interfaz gorda: obliga a TODOS los empleados a "programar"
public interface Empleado {
    void programar();
    void administrarNomina();
}

public class Programador implements Empleado {
    public void programar() { /* ok */ }
    public void administrarNomina() { throw new UnsupportedOperationException(); }
}
```

CONCEPTO — I

Dividiendo el contrato

```
public interface Programador { void programar(); }
public interface Administrador { void administrarNomina(); }

public class DesarrolladorSenior implements Programador {
    public void programar() { /* ... */ }
}
public class LiderDeArea implements Programador, Administrador {
    public void programar() { /* ... */ }
    public void administrarNomina() { /* ... */ }
}
```

Cada clase implementa exactamente las interfaces que le corresponden — ninguna carga con métodos que no usa.

CONCEPTO — D

Inversión de dependencias: depender de abstracciones

Las clases de alto nivel (reglas de negocio) no deberían depender directamente de clases de bajo nivel (detalles concretos, como una base de datos específica) — ambas deberían depender de una **abstracción** (una interfaz). Así, cambiar el detalle concreto no obliga a tocar la regla de negocio.

```
// Depende directamente del detalle concreto
public class ServicioNotificaciones {
    private EmailSender sender = new EmailSender(); // acoplado a Email
}

public interface Notificador { void enviar(String mensaje); }

public class ServicioNotificaciones {
    private Notificador notificador; // depende de la abstracción
    public ServicioNotificaciones(Notificador n) { this.notificador = n; }
}
```

CASO

Por qué esto importa a escala: el "Fail Whale" de Twitter

Entre 2007 y 2010, Twitter era conocido por su famosa "ballena" de error — la plataforma caía constantemente bajo el crecimiento de usuarios. Su arquitectura inicial en Ruby on Rails estaba fuertemente acoplada: cambiar una parte del sistema afectaba muchas otras, dificultando escalar y corregir errores rápido. Twitter migró gradualmente componentes críticos a servicios en la JVM, separados por responsabilidad, lo que le permitió escalar y evolucionar cada parte de forma independiente.

Caso ampliamente documentado en la industria como ejemplo de por qué el desacoplamiento importa a escala.

TRANSICIÓN

De la teoría a cuándo aplicarla

No hace falta ser Twitter para que SOLID importe: el mismo problema de acoplamiento aparece en un proyecto pequeño de curso cuando una clase crece sin control. Pero eso no significa que haya que aplicar los cinco principios a rajatabla desde la primera línea.

TENSIÓN CONCEPTUAL

¿SOLID siempre, desde el primer momento?

Aplicar los cinco principios de forma extrema desde el inicio de un proyecto pequeño puede producir **sobre-ingeniería**: demasiadas interfaces e indirecciones para un problema simple, más difíciles de seguir que el código "sin principios". SOLID resuelve problemas de **cambio y crecimiento** — si una clase nunca va a cambiar ni a tener una segunda implementación, dividirla preventivamente no siempre compensa la complejidad que agrega. El criterio práctico: aplícalos donde ya ves señales de acoplamiento o donde sabes que el código va a evolucionar.

INTERACCIÓN

Detecten la violación

En parejas: revisen esta clase y decidan qué principio(s) de SOLID viola y cómo la dividirían.

```
public class GestorPedidos {  
    public void registrarPedido(String producto, int cantidad) { /* ... */ }  
    public void calcularImpuestos(double monto) { /* ... */ }  
    public void enviarCorreoConfirmacion(String email) { /* ... */ }  
    public void guardarEnBaseDeDatos() { /* ... */ }  
}
```

6 minutos · compartan su propuesta con otra pareja

SÍNTESIS

Lo que hay que llevarse de hoy

1. **S** — cada clase debe tener un único motivo para cambiar.
2. **O** — extiende comportamiento con clases/interfaces nuevas, sin modificar código que ya funciona.
3. **L** — una subclase debe poder sustituir a su clase padre sin sorprender al código que la usa.
4. **I** — prefiere varias interfaces pequeñas y específicas a una interfaz grande y genérica.
5. **D** — depende de abstracciones (interfaces), no de implementaciones concretas.
6. SOLID es una guía para decidir **cuándo** dividir responsabilidades, no una obligación de aplicar los cinco principios en cada clase.



CIERRE

Antes de la próxima sesión

- Revisar un proyecto propio de la Unidad 2 e identificar una clase que viole SRP.
- Traer en mente ese ejemplo: en la próxima sesión se retoma para trabajar código limpio.

Próxima sesión: Código limpio — nombres, funciones y comentarios que hacen el código fácil de leer.

